

```

# tsp_beam_poly2opt_autogen.py
# - Matrix generator: (n, seed, kind)
# - Structural Beam TSP (polynomial-time heuristic) + tiny bounded 2-opt repair (polynomial)
# - Held-Karp exact TSP only for n <= 12 for verification (exponential)
# - Prints beam route/cost/time/memory; prints HK & MATCH/MISMATCH when applicable

import math, random, time, tracemalloc, heapq
from typing import List, Tuple, Optional

# -----
# Matrix generation
# -----
def generate_matrix(n: int, seed: int, kind: str = "asymmetric",
                   wmin: int = 1, wmax: int = 10, block_prob: float = 0.08) -> List[List[float]]:
    """
    kind:
    - "asymmetric": directed, random weights
    - "symmetric" : undirected-like (A[i][j] == A[j][i])
    - "blocked"   : asymmetric with some edges set to INF
    Diagonal is INF (no self loops).
    """
    rng = random.Random(seed)
    INF = float('inf')
    A = [[INF]*n for _ in range(n)]

    if kind == "symmetric":
        for i in range(n):
            for j in range(i+1, n):
                w = rng.randint(wmin, wmax)
                A[i][j] = w
                A[j][i] = w
        # small symmetric blockers (rare)
        for i in range(n):
            for j in range(i+1, n):
                if rng.random() < 0.03:
                    A[i][j] = INF
                    A[j][i] = INF

    elif kind == "blocked":
        # asymmetric with random blockers
        for i in range(n):
            for j in range(n):
                if i == j:
                    continue
                if rng.random() < block_prob:
                    A[i][j] = INF
                else:
                    A[i][j] = rng.randint(wmin, wmax)
        # ensure at least one outgoing edge per node
        for i in range(n):
            if all(math.isinf(A[i][j]) for j in range(n)):
                j = rng.randrange(n)
                while j == i:
                    j = rng.randrange(n)
                A[i][j] = rng.randint(wmin, wmax)

```

```

else: # asymmetric
    for i in range(n):
        for j in range(n):
            if i == j:
                continue
            A[i][j] = rng.randint(wmin, wmax)

    return A

# -----
# Structural Beam + bounded 2-opt (polynomial)
# -----
def tsp_structural_beam(matrix: List[List[float]],
                        K: int = 5, L: int = 6, TAU: float = 0.30, M: int = 15,
                        MAX_2OPT_PASSES: int = 2) -> Tuple[Optional[int], Optional[List[int]]]:
    """
    Returns (best_cost:int, best_route_1based:List[int]) or (None, None) if no tour.
    Beam + small bounded 2-opt repair (keeps time & space polynomial).
    """
    n = len(matrix)
    INF = 10**12

    def blocked(i: int, j: int) -> bool:
        return i == j or matrix[i][j] == float('inf')

    # ----- helpers for bounded 2-opt -----
    def tour_cost(tour: List[int]) -> float:
        s = 0.0
        for a, b in zip(tour, tour[1:]):
            s += matrix[a][b]
        s += matrix[tour[-1]][tour[0]]
        return s

    def two_opt_repair(tour: List[int], max_passes: int = MAX_2OPT_PASSES) -> Tuple[List[int], float]:
        """Up to max_passes improving 2-opt moves; 0(max_passes * n^2)."""
        nloc = len(tour)

        def two_opt_once(cur: List[int]) -> Tuple[List[int], float, bool]:
            cur_cost = tour_cost(cur)
            for i in range(nloc - 1):
                a, b = cur[i], cur[(i+1) % nloc]
                for k in range(i+2, nloc - (0 if i > 0 else 1)): # avoid adjacent & closing edge
                    # We must fully evaluate the new tour cost because reversing segments
                    # in asymmetric/blocked graphs changes the internal edge weights!
                    nxt = cur[:i+1] + list(reversed(cur[i+1:k+1])) + cur[k+1:]
                    new_cost = tour_cost(nxt)
                    if new_cost < cur_cost - 1e-12:
                        return nxt, new_cost, True
            return cur, cur_cost, False

        cur = tour[:]
        cur_cost = tour_cost(cur)
        for _ in range(max_passes):
            cur, cur_cost, improved = two_opt_once(cur)
            if not improved:
                break
        return cur, cur_cost

```

```

# ===== precompute outgoing neighbors =====
out_neighbors = []
for i in range(n):
    lst = [(j, matrix[i][j]) for j in range(n) if not blocked(i, j)]
    lst.sort(key=lambda x: x[1])
    out_neighbors.append(lst)

rank_out = [None]*n for _ in range(n)]
best_out = [None]*n
for i in range(n):
    for r, (j, w) in enumerate(out_neighbors[i]):
        rank_out[i][j] = r + 1
    if out_neighbors[i]:
        best_out[i] = out_neighbors[i][0]

incoming_anchor_sources = [[] for _ in range(n)]
for i in range(n):
    if out_neighbors[i]:
        tgt = out_neighbors[i][0][0]
        incoming_anchor_sources[tgt].append(i)
for j in range(n):
    incoming_anchor_sources[j].sort(key=lambda x: matrix[x][j])

in_rank_best = [INF]*n
for j in range(n):
    best = INF
    for i in range(n):
        if i != j and not blocked(i, j) and matrix[i][j] < best:
            best = matrix[i][j]
    in_rank_best[j] = best if best < INF else INF

def cheap_lower_bound(unvisited: set, left: int, right: int) -> float:
    lb = 0.0
    for u in unvisited:
        best = INF
        for v, w in out_neighbors[u]:
            if v in unvisited or v == left or v == right:
                if w < best:
                    best = w
        lb += 0 if best == INF else best
    best_r = INF
    for v, w in out_neighbors[right]:
        if v in unvisited or v == left:
            if w < best_r:
                best_r = w
    lb += 0 if best_r == INF else best_r
    best_in = INF
    for src in incoming_anchor_sources[left]:
        if src in unvisited and matrix[src][left] < best_in:
            best_in = matrix[src][left]
    if best_in < INF:
        lb += best_in
    return lb

def suspicion(u: int, v: int) -> int:
    s = 0
    if rank_out[u][v] and rank_out[u][v] <= 2:
        s += 1
    if best_out[u] and best_out[u][0] == v:

```

```

if best_out[u] and best_out[v] == v:
    s += 1
if u in incoming_anchor_sources[v]:
    s += 1
if matrix[u][v] <= in_rank_best[v]:
    s += 1
return s

def two_step_trap_right(right: int, used_set: set) -> bool:
    feas = [(v, w) for (v, w) in out_neighbors[right] if v not in used_set]
    if len(feas) < 2:
        return False
    limit = min(len(feas), K + L)
    vals = []
    for idx in range(limit):
        v, w = feas[idx]
        best2 = 10**12
        for v2, w2 in out_neighbors[v]:
            if v2 not in used_set and v2 != right and w2 < best2:
                best2 = w2
        vals.append(w + (0 if best2 == 10**12 else best2))
    m = min(vals)
    return any(val <= 0.98 * m for val in vals[1:])

def two_step_trap_left(left: int, used_set: set) -> bool:
    sracs = [x for x in incoming_anchor_sources[left] if x not in used_set]
    if len(sracs) < 2:
        return False
    limit = min(len(sracs), K + L)
    vals = []
    for idx in range(limit):
        u = sracs[idx]
        w = matrix[u][left]
        best2 = 10**12
        for v2, w2 in out_neighbors[u]:
            if v2 not in used_set and v2 != left and w2 < best2:
                best2 = w2
        vals.append(w + (0 if best2 == 10**12 else best2))
    m = min(vals)
    return any(val <= 0.98 * m for val in vals[1:])

anchors = []
for i in range(n):
    if best_out[i] is not None:
        j, w = best_out[i]
        anchors.append((w, i, j))
anchors.sort()

best_overall = (INF, None) # (cost, tour_nodes_0based)
used_anchor_edge = set()

def grow_from_seed(i0: int, j0: int):
    nonlocal best_overall, used_anchor_edge
    left, right = i0, j0
    used = frozenset({i0, j0})
    cur_cost = matrix[i0][j0]
    pL = (i0,) # reversed
    pR = (j0,) # forward
    heap = []

```

```

heapq.heappush(heap, (cur_cost, cur_cost, left, right, used, pL, pR))

while heap:
    batch = []
    for _ in range(min(len(heap), M)):
        batch.append(heapq.heappop(heap))
    new_pool = []

    for heur, cur, left, right, used_f, pL, pR in batch:
        used_set = set(used_f)

        # try close and repair
        if len(used_set) == n and not blocked(right, left):
            base_tour = list(pL[:-1]) + list(pR)
            base_cost = cur + matrix[right][left]
            repaired, repaired_cost = two_opt_repair(base_tour, max_passes=MAX_20PT_PASSES)
            if repaired_cost < base_cost:
                total = repaired_cost
                final_tour = repaired
            else:
                total = base_cost
                final_tour = base_tour
            if total < best_overall[0]:
                best_overall = (total, final_tour)
                for a, b in zip(final_tour, final_tour[1:] + [final_tour[0]]):
                    if out_neighbors[a] and out_neighbors[a][0][0] == b:
                        used_anchor_edge.add((a, b))
            continue

        # right side
        feasR = [(v, w) for (v, w) in out_neighbors[right] if v not in used_set]
        r_mand = feasR[:K]
        r_extra = feasR[K:K+L] if len(feasR) > K else []
        openR = False
        if len(feasR) >= 2:
            w1 = feasR[0][1]; w2 = feasR[1][1]
            if w1 > 0 and (w2 - w1) / w1 <= TAU:
                openR = True
        unvisited = [u for u in range(n) if u not in used_set]
        if best_overall[0] < INF:
            lb = cheap_lower_bound(set(unvisited), left, right)
            if cur + lb >= 0.95 * best_overall[0]:
                openR = True
        for (v, w) in r_extra:
            if suspicion(right, v) >= 2:
                openR = True; break
        if two_step_trap_right(right, used_set):
            openR = True
        r_cands = r_mand + (r_extra if openR else [])

        # left side
        sources = [x for x in incoming_anchor_sources[left] if x not in used_set]
        l_mand = [(x, matrix[x][left]) for x in sources[:K]]
        l_extra = [(x, matrix[x][left]) for x in sources[K:K+L]] if len(sources) > K else []
        openL = False
        if len(sources) >= 2:
            w1 = matrix[sources[0]][left]; w2 = matrix[sources[1]][left]
            if w1 > 0 and (w2 - w1) / w1 <= TAU:

```

```

        openL = True
    if best_overall[0] < INF:
        lb = cheap_lower_bound(set(unvisited), left, right)
        if cur + lb >= 0.95 * best_overall[0]:
            openL = True
    for (x, w) in l_extra:
        if suspicion(x, left) >= 2:
            openL = True; break
    if two_step_trap_left(left, used_set):
        openL = True
    l_cands = l_mand + (l_extra if openL else [])

# bounded escape if both empty
if not r_cands and not l_cands:
    extra_r = None
    for v, w in out_neighbors[right]:
        if v not in used_set:
            extra_r = (v, w); break
    extra_l = None
    best_w_in, best_u = INF, None
    for u in range(n):
        if u not in used_set and not blocked(u, left):
            if matrix[u][left] < best_w_in:
                best_w_in, best_u = matrix[u][left], u
    if best_u is not None:
        extra_l = (best_u, best_w_in)
    if extra_r is None and extra_l is None:
        continue
    if extra_r is not None:
        r_cands = [extra_r]
    if extra_l is not None:
        l_cands = [extra_l]

# expand right
for (v, w) in r_cands:
    new_used = set(used_set); new_used.add(v)
    new_pL = pL; new_pR = pR + (v,)
    new_left = left; new_right = v
    new_cur = cur + w
    unvis2 = [u for u in range(n) if u not in new_used]
    lb = cheap_lower_bound(set(unvis2), new_left, new_right)
    heur2 = new_cur + lb
    heapq.heappush(new_pool, (heur2, new_cur, new_left, new_right,
                              frozenset(new_used), new_pL, new_pR))

# expand left
for (x, w) in l_cands:
    new_used = set(used_set); new_used.add(x)
    new_pL = pL + (x,)
    new_pR = pR
    new_left = x; new_right = right
    new_cur = cur + w
    unvis2 = [u for u in range(n) if u not in new_used]
    lb = cheap_lower_bound(set(unvis2), new_left, new_right)
    heur2 = new_cur + lb
    heapq.heappush(new_pool, (heur2, new_cur, new_left, new_right,
                              frozenset(new_used), new_pL, new_pR))

```

```

        if not new_pool:
            heap = []
            break
        new_pool.sort(key=lambda x: x[0])
        heap = new_pool[:M]

    for _, i, j in anchors:
        if blocked(i, j):
            continue
        grow_from_seed(i, j)

    if best_overall[1] is None:
        return None, None
    route0 = best_overall[1]
    route1 = [x+1 for x in route0]
    if route1[-1] != route1[0]:
        route1.append(route1[0])
    return int(round(best_overall[0])), route1

# -----
# Held-Karp exact TSP (verification only; exponential)
# -----
def held_karp_tsp(matrix: List[List[float]]) -> Tuple[Optional[int], Optional[List[int]]]:
    n = len(matrix)
    INF = float('inf')
    dp = {}
    for j in range(1, n):
        w = matrix[0][j]
        if math.isinf(w):
            continue
        dp[(1<<0)|(1<<j), j] = (w, 0)

    for mask in range(1<<n):
        if not (mask & 1):
            continue
        for j in range(n):
            if j == 0 or not (mask & (1<<j)):
                continue
            if (mask, j) not in dp:
                continue
            cost_j, prev_j = dp[(mask, j)]
            for k in range(n):
                if (mask & (1<<k)) or math.isinf(matrix[j][k]):
                    continue
                nmask = mask | (1<<k)
                cand = cost_j + matrix[j][k]
                key = (nmask, k)
                if key not in dp or cand < dp[key][0]:
                    dp[key] = (cand, j)

    full = (1<<n) - 1
    best = None
    best_end = None
    for j in range(1, n):
        if (full, j) in dp and not math.isinf(matrix[j][0]):
            cand = dp[(full, j)][0] + matrix[j][0]
            if best is None or cand < best:
                best = cand

```

```

        best_end = j

    if best is None:
        return None, None

    tour = [0]
    mask = full
    j = best_end
    path_rev = [j]
    while j != 0:
        cost_j, prev_j = dp[(mask, j)]
        mask ^= (1<<j)
        j = prev_j
        if j != 0:
            path_rev.append(j)
    path_rev.reverse()
    tour.extend(path_rev)
    tour.append(0)
    return int(best), [x+1 for x in tour]

# -----
# Experiment runner
# -----
def run_case(n: int, seed: int, kind: str,
             K=5, L=6, TAU=0.30, M=25, MAX_20PT_PASSES=2, hk_limit: int = 10):
    print(f"\n=== n={n}, seed={seed}, kind={kind} ===")
    mat = generate_matrix(n, seed, kind)

    # Beam timing/memory ONLY
    tracemalloc.start()
    t0 = time.perf_counter()
    b_cost, b_route = tsp_structural_beam(mat, K=K, L=L, TAU=TAU, M=M, MAX_20PT_PASSES=MAX_20PT_PASSES)
    t1 = time.perf_counter()
    cur, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    beam_time = t1 - t0
    beam_mem_mb = peak / (1024*1024)

    if b_route is None:
        print("Beam: No complete tour found.")
    else:
        print(f"Beam cost : {b_cost}")
        print("Beam route:", " -> ".join(map(str, b_route)))
    print(f"Beam time : {beam_time:.6f}s | Peak mem: {beam_mem_mb:.3f} MB")

    # Verification
    if n <= hk_limit:
        hk_cost, hk_route = held_karp_tsp(mat)
        if hk_cost is None:
            print("HK: No Hamiltonian cycle exists (given blocks).")
            verified = (b_route is None)
        else:
            print(f"HK cost : {hk_cost}")
            print("HK route :", " -> ".join(map(str, hk_route)))
            verified = (b_cost <= hk_cost) if b_cost is not None else False

        if verified:
            print("✅ MATCH - beam found optimal tour (HK verified).")
        else:

```



```

    print("✗ MISMATCH – investigate. Matrix follows:")
    for row in mat:
        print(" ", ["INF" if math.isinf(x) else int(x) for x in row])
    else:
        print(f"(HK skipped for n={n}; limit={hk_limit})")

# -----
# Main: edit CASES and beam params below
# -----
if __name__ == "__main__":
    # You can list many cases here for a "stress test"
    CASES = [
        (10, 2, "symmetric"),
        (10, 2, "asymmetric"),
        (10, 2, "blocked"),
        (25, 2, "symmetric"), # HK still runs
        (25, 2, "asymmetric"), # HK still runs
        (25, 2, "blocked"), # HK still runs
        (50, 2, "symmetric"), # HK skipped , as above limit
        (50, 2, "asymmetric"), # HK skipped , as above limit
        (50, 2, "blocked"), # HK Skipped , as above limit
        (100, 2, "symmetric"), # HK skipped , as above limit
        (100, 2, "asymmetric"), # HK skipped , as above limit
        (100, 2, "blocked") # HK skipped , as above limit
    ]

    # Beam parameters: Increased M to 25 to allow wider search around INF traps in blocked matrices
    K, L, TAU, M, MAX_20PT_PASSES = 5, 6, 0.30, 25, 2
    HK_LIMIT = 10

    for n, seed, kind in CASES:
        run_case(n, seed, kind, K=K, L=L, TAU=TAU, M=M, MAX_20PT_PASSES=MAX_20PT_PASSES, hk_limit=HK_LIMIT)

```

```

=== n=10, seed=2, kind=symmetric ===
Beam cost : 22
Beam route: 1 -> 2 -> 4 -> 10 -> 5 -> 7 -> 6 -> 9 -> 3 -> 8 -> 1
Beam time : 1.049417s | Peak mem: 0.142 MB
HK cost : 22
HK route : 1 -> 8 -> 3 -> 9 -> 6 -> 7 -> 5 -> 10 -> 4 -> 2 -> 1
✅ MATCH – beam found optimal tour (HK verified).

=== n=10, seed=2, kind=asymmetric ===
Beam cost : 32
Beam route: 1 -> 2 -> 3 -> 5 -> 10 -> 7 -> 4 -> 9 -> 6 -> 8 -> 1
Beam time : 1.075994s | Peak mem: 0.106 MB
HK cost : 32
HK route : 1 -> 4 -> 9 -> 2 -> 3 -> 5 -> 10 -> 7 -> 6 -> 8 -> 1
✅ MATCH – beam found optimal tour (HK verified).

=== n=10, seed=2, kind=blocked ===
Beam cost : 23
Beam route: 3 -> 2 -> 1 -> 4 -> 10 -> 8 -> 5 -> 7 -> 6 -> 9 -> 3
Beam time : 0.994114s | Peak mem: 0.095 MB
HK cost : 23
HK route : 1 -> 4 -> 10 -> 8 -> 5 -> 7 -> 6 -> 9 -> 3 -> 2 -> 1
✅ MATCH – beam found optimal tour (HK verified).

=== n=25, seed=2, kind=symmetric ===

```

```
Beam cost : 35
Beam route: 7 -> 15 -> 22 -> 20 -> 11 -> 8 -> 6 -> 18 -> 10 -> 5 -> 25 -> 9 -> 4 -> 1 -> 3 -> 14 -> 2 -> 13 -> 24 -> 12 -> 19 -> 16 -> 21 -> 17 -> 23 -> 7
Beam time : 22.215917s | Peak mem: 0.715 MB
(HK skipped for n=25; limit=10)

=== n=25, seed=2, kind=asymmetric ===
Beam cost : 34
Beam route: 17 -> 1 -> 23 -> 20 -> 18 -> 14 -> 21 -> 4 -> 24 -> 8 -> 11 -> 19 -> 9 -> 25 -> 13 -> 10 -> 5 -> 15 -> 6 -> 3 -> 7 -> 16 -> 2 -> 12 -> 22 -> 17
Beam time : 25.995278s | Peak mem: 0.382 MB
(HK skipped for n=25; limit=10)

=== n=25, seed=2, kind=blocked ===
Beam cost : 36
Beam route: 17 -> 10 -> 1 -> 2 -> 24 -> 12 -> 21 -> 25 -> 6 -> 3 -> 13 -> 4 -> 18 -> 23 -> 22 -> 19 -> 11 -> 9 -> 15 -> 8 -> 7 -> 20 -> 16 -> 5 -> 14 -> 17
Beam time : 25.184720s | Peak mem: 0.295 MB
(HK skipped for n=25; limit=10)

=== n=50, seed=2, kind=symmetric ===
Beam cost : 55
Beam route: 34 -> 16 -> 3 -> 40 -> 4 -> 38 -> 10 -> 27 -> 20 -> 22 -> 17 -> 19 -> 12 -> 1 -> 24 -> 31 -> 5 -> 21 -> 43 -> 26 -> 7 -> 6 -> 23 -> 33 -> 28 -> 39 -> 35 -> 49 -> 4
Beam time : 180.785757s | Peak mem: 0.956 MB
(HK skipped for n=50; limit=10)

=== n=50, seed=2, kind=asymmetric ===
Beam cost : 52
Beam route: 21 -> 3 -> 37 -> 4 -> 32 -> 5 -> 11 -> 1 -> 12 -> 22 -> 6 -> 26 -> 18 -> 25 -> 17 -> 29 -> 28 -> 45 -> 14 -> 30 -> 16 -> 33 -> 40 -> 10 -> 7 -> 39 -> 46 -> 35 -> 9
Beam time : 268.392248s | Peak mem: 0.877 MB
(HK skipped for n=50; limit=10)

=== n=50, seed=2, kind=blocked ===
Beam cost : 55
Beam route: 21 -> 3 -> 37 -> 4 -> 32 -> 5 -> 11 -> 1 -> 12 -> 22 -> 6 -> 26 -> 18 -> 25 -> 17 -> 29 -> 28 -> 45 -> 14 -> 30 -> 16 -> 33 -> 40 -> 10 -> 7 -> 39 -> 46 -> 35 -> 9
```